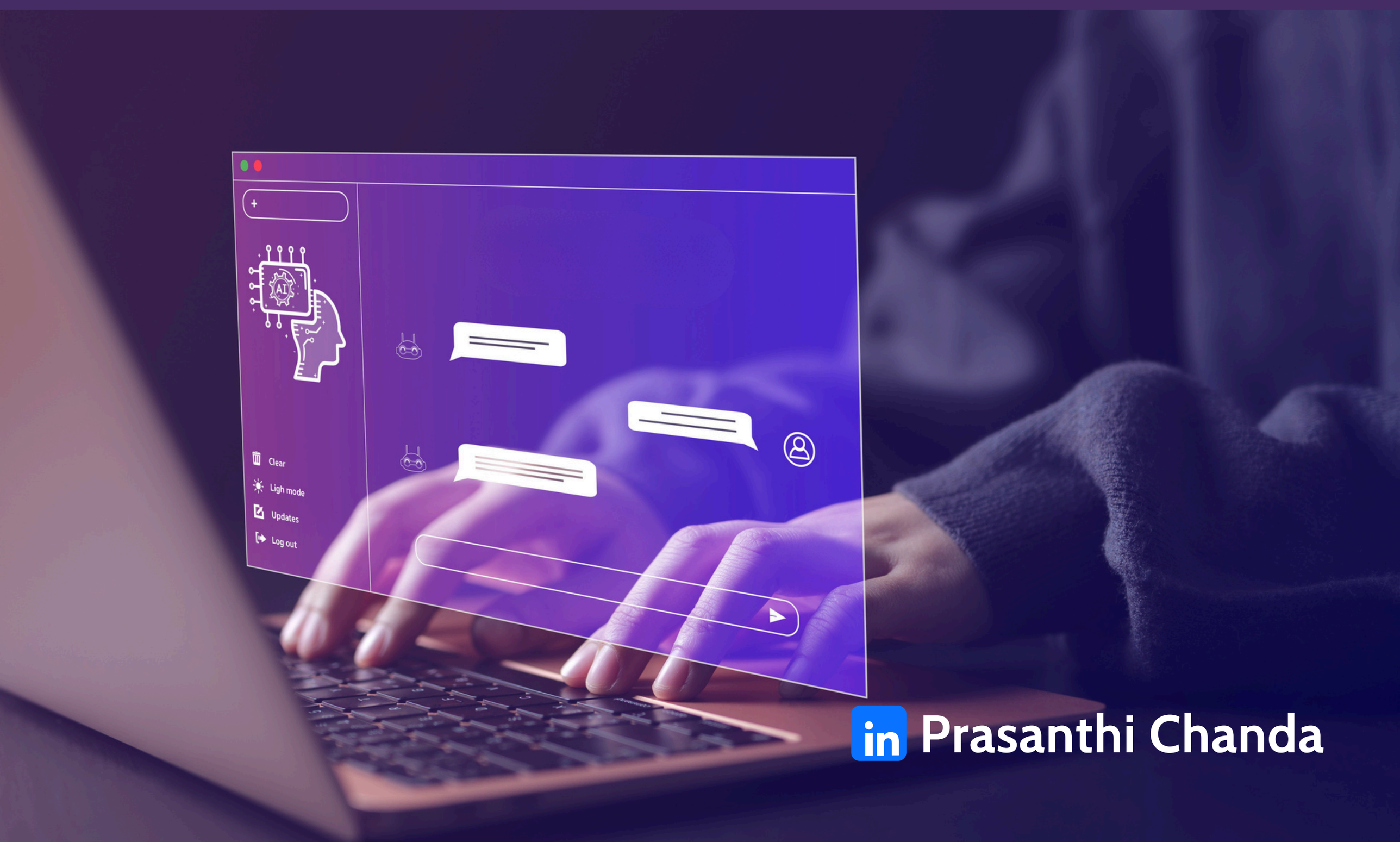
PROV LOGIC

TOP 30 ADVANCED CASE BASED INTERVIEW QUESTIONS FROM TOP PRODUCT COMPANIES

APPLE, QUALCOMM, INTEL, AMD, ARM, . .



Prasanthi Chanda

CASE-BASED INTERVIEW QUESTION 1

You're designing an asynchronous FIFO to transfer data between two different clock domains — a 100 MHz write clock and a 50 MHz read clock. How would you handle clock domain crossing, avoid metastability, and ensure reliable data transfer? Provide the Verilog code for your implementation.

Asked in: Qualcomm, Apple, Intel, NVIDIA, Broadcom

SOLUTION STRATEGY:

1. **Use Gray code pointers to prevent glitches during pointer transfer.**
2. **Use a dual-port memory array for simultaneous read and write.**
3. **Synchronize the write pointer to the read domain and the read pointer to the write domain using 2-stage flip-flops.**
4. **Compare pointers safely in respective domains to generate full and empty flags.**

```
module async_fifo #(
    parameter ADDR_WIDTH = 4,
    parameter DATA_WIDTH = 8
)(
    input  wire                  wr_clk,
    input  wire                  rd_clk,
    input  wire                  rst,
    input  wire                  wr_en,
    input  wire                  rd_en,
    input  wire [DATA_WIDTH-1:0] wr_data,
    output reg  [DATA_WIDTH-1:0] rd_data,
    output wire                  full,
    output wire                  empty
);

    localparam DEPTH = (1 << ADDR_WIDTH);
```

```
// FIFO memory
 reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

 // Binary and Gray code pointers
 reg [ADDR_WIDTH:0] wr_ptr_bin, rd_ptr_bin;
 reg [ADDR_WIDTH:0] wr_ptr_gray, rd_ptr_gray;

// Synchronized pointers across domains
reg [ADDR_WIDTH:0] wr_ptr_gray_sync1, wr_ptr_gray_sync2;
reg [ADDR_WIDTH:0] rd_ptr_gray_sync1, rd_ptr_gray_sync2;

// Write logic
always @(posedge wr_clk or posedge rst) begin
    if (rst) begin
        wr_ptr_bin  <= 0;
        wr_ptr_gray <= 0;
    end else if (wr_en && !full) begin
        mem[wr_ptr_bin[ADDR_WIDTH-1:0]] <= wr_data;
        wr_ptr_bin  <= wr_ptr_bin + 1;
        wr_ptr_gray <= (wr_ptr_bin + 1) ^ ((wr_ptr_bin + 1) >> 1);
    end
end

// Read logic
always @(posedge rd_clk or posedge rst) begin
    if (rst) begin
        rd_ptr_bin  <= 0;
        rd_ptr_gray <= 0;
        rd_data     <= 0;
    end else if (rd_en && !empty) begin
        rd_data     <= mem[rd_ptr_bin[ADDR_WIDTH-1:0]];
        rd_ptr_bin  <= rd_ptr_bin + 1;
        rd_ptr_gray <= (rd_ptr_bin + 1) ^ ((rd_ptr_bin + 1) >> 1);
    end
end
```

```
// Synchronize read pointer to write clock domain
always @(posedge wr_clk or posedge rst) begin
    if (rst) begin
        rd_ptr_gray_sync1 <= 0;
        rd_ptr_gray_sync2 <= 0;
    end else begin
        rd_ptr_gray_sync1 <= rd_ptr_gray;
        rd_ptr_gray_sync2 <= rd_ptr_gray_sync1;
    end
end

// Synchronize write pointer to read clock domain
always @(posedge rd_clk or posedge rst) begin
    if (rst) begin
        wr_ptr_gray_sync1 <= 0;
        wr_ptr_gray_sync2 <= 0;
    end else begin
        wr_ptr_gray_sync1 <= wr_ptr_gray;
        wr_ptr_gray_sync2 <= wr_ptr_gray_sync1;
      end
    end

    // Full condition logic
    assign full = (wr_ptr_gray ==
{~rd_ptr_gray_sync2[ADDR_WIDTH:ADDR_WIDTH-1],
                          rd_ptr_gray_sync2[ADDR_WIDTH-2:0]});

    // Empty condition logic
    assign empty = (rd_ptr_gray == wr_ptr_gray_sync2);

endmodule
```

CASE-BASED INTERVIEW QUESTION 2

You are tasked with designing a clock gating mechanism to reduce dynamic power consumption in a digital design. How would you implement clock gating in Verilog to achieve this, and what are the key points to ensure it's implemented correctly?

Asked in: Apple, Qualcomm, Intel, NVIDIA, AMD

SOLUTION STRATEGY:

1. **Use AND gate-based clock gating to selectively disable the clock when the module isn't active.**
2. **Ensure that the clock enable signal is synchronized with the clock domain to prevent glitches.**
3. **Design the gating logic to only disable the clock when the module is idle to save power.**

```
module clock_gating (
    input wire clk,             // Input clock
    input wire enable,          // Clock enable signal
    output wire gated_clk       // Gated clock output
);

    reg gated_clk_int;

    // Clock gating logic
    always @(clk or enable) begin
        if (enable)
            gated_clk_int = clk;   // Pass the clock when enabled
        else
            gated_clk_int = 1'b0; // Disable clock when not enabled
    end

    assign gated_clk = gated_clk_int;

endmodule
```

CASE-BASED INTERVIEW QUESTION 3

You are designing a priority encoder in Verilog for a system that has 8 inputs and outputs the binary representation of the highest priority active input. If no input is active, the encoder should output a default value of 3'b111. How would you design this system?

Asked in: NVIDIA, Broadcom, Intel, Qualcomm

SOLUTION STRATEGY:

1. **Use a priority encoder design that checks the highest active bit and outputs the corresponding binary value.**
2. **Ensure that if no input is active, the output is set to a default value (3'b111).**
3. **Use a case statement or if-else logic to implement the priority encoder functionality.**

```
module priority_encoder (
    input wire [7:0] in,   // 8-bit input
    output reg [2:0] out   // 3-bit output (binary representation)
);

    always @(*) begin
        casez (in)
            8'b1???????: out = 3'b000;  // Highest priority is bit 7
            8'b01??????: out = 3'b001;  // Highest priority is bit 6
            8'b001?????: out = 3'b010;  // Highest priority is bit 5
            8'b0001????: out = 3'b011;  // Highest priority is bit 4
            8'b00001??? : out = 3'b100; // Highest priority is bit 3
            8'b000001??: out = 3'b101;  // Highest priority is bit 2
            8'b0000001?: out = 3'b110;  // Highest priority is bit 1
            8'b00000001: out = 3'b111;  // Highest priority is bit 0
            default: out = 3'b111;      // Default case when no bit is set
        endcase
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 4

You are designing an arbiter for a system where multiple masters are requesting access to a shared resource. The arbiter must provide fair access to the resource and prioritize requests based on round-robin scheduling. Write the Verilog code for such an arbiter.

Asked in: Qualcomm, Broadcom, AMD, Intel

SOLUTION STRATEGY:

1. **Implement a round-robin scheduling mechanism to ensure fair access to the shared resource.**
2. **Use a priority encoder to select which master gets access based on the round-robin policy.**
3. **Ensure that the arbiter can handle multiple requests and provide access in a fair manner without starvation.**

```
module round_robin_arbiter (
    input wire clk,              // Clock signal
    input wire rst,              // Reset signal
    input wire [3:0] request,    // 4-bit master request input
    output reg [3:0] grant       // 4-bit master grant output
);

    reg [3:0] priority;          // Round-robin priority register
    reg [3:0] next_priority;     // Next priority register

    always @(posedge clk or posedge rst) begin
        if (rst)
            priority <= 4'b0001;  // Start with the lowest priority (Master 0)
        else
            priority <= next_priority;
    end
```

```
always @(*) begin
    next_priority = priority;
    case (priority)
        4'b0001: next_priority = (request[1]) ? 4'b0010 : 4'b0001;
        4'b0010: next_priority = (request[2]) ? 4'b0100 : 4'b0010;
        4'b0100: next_priority = (request[3]) ? 4'b1000 : 4'b0100;
        4'b1000: next_priority = (request[0]) ? 4'b0001 : 4'b1000;
        default: next_priority = 4'b0001;
    endcase
end

always @(*) begin
    case (priority)
        4'b0001: grant = request[0] ? 4'b0001 : 4'b0000;
        4'b0010: grant = request[1] ? 4'b0010 : 4'b0000;
        4'b0100: grant = request[2] ? 4'b0100 : 4'b0000;
        4'b1000: grant = request[3] ? 4'b1000 : 4'b0000;
        default: grant = 4'b0000;
    endcase
end

endmodule
```

CASE-BASED INTERVIEW QUESTION 5

You are asked to implement a synchronous reset in a design that should reset multiple flip-flops simultaneously when the reset signal is asserted. The reset signal should be active-low. Provide a Verilog solution for this requirement.

Asked in: Apple, Intel, AMD, NVIDIA

SOLUTION STRATEGY:

1. **Use active-low reset logic to synchronize the reset signal across flip-flops.**
2. **Ensure all flip-flops in the design are reset simultaneously when the reset signal is asserted.**

```
module sync_reset (
    input wire clk,        // Clock signal
    input wire nreset,     // Active-low reset signal
    output reg [3:0] q     // 4-bit output
);

    always @(posedge clk or negedge nreset) begin
        if (~nreset)          // When reset is low
            q <= 4'b0000;     // Reset all outputs to 0
        else
            q <= q + 1;       // Otherwise, increment q
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 6

You are designing a low-power RTL system and need to implement clock gating with conditional logic to ensure that clocking only happens when the module is required to perform an operation. How would you implement clock gating in Verilog while minimizing power consumption?

Asked in: Intel, AMD, Qualcomm, Apple

SOLUTION STRATEGY:

1. **Use a combination of AND gates to conditionally gate the clock.**
2. **Ensure that the enable signal accurately reflects whether the module is active, and disable the clock when not needed.**
3. **Properly synchronize the enable signal to prevent glitches.**

```
module low_power_clock_gating (
    input wire clk,              // Clock input
    input wire enable,           // Conditional enable signal
    output wire gated_clk        // Gated clock output
);

    reg gated_clk_int;

    // Clock gating logic
    always @(clk or enable) begin
        if (enable)
            gated_clk_int = clk;    // Allow clock to pass when enabled
        else
            gated_clk_int = 1'b0;  // Disable clock when not enabled
    end

    assign gated_clk = gated_clk_int;

endmodule
```

CASE-BASED INTERVIEW QUESTION 7

Design an LFSR (Linear Feedback Shift Register) in Verilog that can generate a 16-bit pseudo-random sequence. The LFSR should use a feedback polynomial of $x^{16} + x^{14} + x^{13} + x^{11} + 1$. Provide the Verilog code for the LFSR implementation.

Asked in: Qualcomm, Apple, NVIDIA, Broadcom

SOLUTION STRATEGY:

1. **Implement the LFSR shift register using the specified polynomial for feedback.**
2. **Use the XOR gates to combine the tapped bits for feedback generation.**
3. **Ensure that the register continuously shifts with the clock.**

```
module lfsr (
    input wire clk,             // Clock input
    input wire rst,             // Active-low reset
    output reg [15:0] lfsr_out   // 16-bit LFSR output
);

    // Feedback polynomial: x^16 + x^14 + x^13 + x^11 + 1
    always @(posedge clk or negedge rst) begin
        if (~rst)
            lfsr_out <= 16'hFFFF;  // Initialize the LFSR to all 1's
        else
            lfsr_out <= {lfsr_out[14:0], lfsr_out[15] ^ lfsr_out[13] ^
lfsr_out[12] ^ lfsr_out[10]};  // Shift and feedback
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 8

You are designing a digital debouncer in Verilog to clean up a noisy input signal from a mechanical switch. The debouncer should ensure that the output changes only once per stable press of the switch. Write the Verilog code for this digital debouncer.

Asked in: Qualcomm, Intel, AMD, Broadcom

SOLUTION STRATEGY:

1. **Use a counter-based debounce mechanism to ensure stable switching.**
2. **After detecting a change in the switch state, the counter should wait for a stable period before allowing a state change in the output.**
3. **Prevent multiple transitions within a short time window.**

```
module debouncer (
    input wire clk,             // Clock input
    input wire rst,             // Active-low reset
    input wire switch_in,       // Noisy switch input
    output reg switch_out       // Debounced output
);

    reg [15:0] counter;         // 16-bit counter for debounce timing
    reg switch_in_reg, switch_in_reg2;

    always @(posedge clk or negedge rst) begin
    if (~rst) begin
        counter <= 0;
        switch_in_reg <= 0;
        switch_in_reg2 <= 0;
        switch_out <= 0;
    end
    else begin
```

```
// Synchronize the input to the clock domain
switch_in_reg2 <= switch_in_reg;
switch_in_reg <= switch_in;

// If the switch has changed state, start the debounce counter
if (switch_in_reg != switch_in_reg2) begin
counter <= 16'h0000;
end else if (counter < 16'hFFFF) begin
counter <= counter + 1;
end

// Output the debounced state when the counter is fully incremented
if (counter == 16'hFFFF) begin
switch_out <= switch_in_reg;
end
end
end

endmodule
```

CASE-BASED INTERVIEW QUESTION 9

Design a 2-bit synchronous up/down counter in Verilog that has a count enable (CE) input and a direction input (up or down). When the direction input is high, the counter counts up; when it's low, the counter counts down. Provide the Verilog code for the counter.

Asked in: Apple, Qualcomm, Intel, Broadcom

SOLUTION STRATEGY:

1. **Use a 2-bit counter and implement the up/down counting mechanism.**
2. **Ensure that the counter increments or decrements based on the direction signal.**
3. **Include a count enable (CE) signal that allows counting only when enabled.**

```
module up_down_counter (
    input wire clk,           // Clock input
    input wire rst,           // Active-low reset
    input wire ce,            // Count enable input
    input wire direction,     // Direction input (1 = up, 0 = down)
    output reg [1:0] count    // 2-bit counter output
);

always @(posedge clk or negedge rst) begin
    if (~rst)
        count <= 2'b00;    // Reset counter to 0
    else if (ce) begin
        if (direction)     // Count up
            count <= count + 1;
        else               // Count down
            count <= count - 1;
    end
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 10

You need to design a pulse width modulator (PWM) that generates a PWM signal with a duty cycle determined by a 4-bit input value. The PWM signal should be generated with a clock period of 10ns. Provide the Verilog code for the PWM generator.

Asked in: Apple, Qualcomm, Intel, NVIDIA

SOLUTION STRATEGY:

1. **Use a counter to count the clock cycles and compare it with the duty cycle input.**
2. **The PWM signal should be high for the number of cycles proportional to the duty cycle and low for the remaining cycles.**

```
module pwm_generator (
    input wire clk,              // Clock input
    input wire [3:0] duty_cycle,  // 4-bit duty cycle input (0 to 15)
    output reg pwm_out           // PWM output signal
);

    reg [3:0] counter;           // 4-bit counter

    always @(posedge clk) begin
        counter <= counter + 1;    // Increment counter on each clock
cycle

        // Compare counter with duty cycle to generate PWM
        if (counter < duty_cycle)
            pwm_out <= 1'b1;     // PWM high
        else
            pwm_out <= 1'b0;     // PWM low
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 11

Design a circular buffer in Verilog with read and write pointers that operate in different clock domains. The buffer should be implemented using dual-port RAM and should handle the read/write operation safely using synchronization techniques.

Asked in: Qualcomm, Broadcom, Intel, AMD

SOLUTION STRATEGY:

1. **Use dual-port RAM to handle both read and write operations.**
2. **Implement synchronization of the write pointer in the read clock domain to prevent metastability.**
3. **Use Gray code or two-flop synchronization for pointer transfer.**

```
module circular_buffer (
    input wire wr_clk,          // Write clock input
    input wire rd_clk,          // Read clock input
    input wire rst,             // Active-low reset
    input wire [7:0] wr_data,   // Write data input
    input wire wr_en,           // Write enable input
    input wire rd_en,           // Read enable input
    output reg [7:0] rd_data    // Read data output
);

    // Dual-port RAM (synchronous to respective clocks)
    reg [7:0] mem [0:15];  // 16x8-bit memory

    // Write pointer (synchronous to wr_clk)
    reg [3:0] wr_ptr;

    // Read pointer (synchronous to rd_clk)
    reg [3:0] rd_ptr;

// Synchronizing the write pointer to the read clock domain
reg [3:0] wr_ptr_sync1, wr_ptr_sync2;

// Write operation
always @(posedge wr_clk or negedge rst) begin
    if (~rst)
        wr_ptr <= 4'b0000;   // Reset write pointer
    else if (wr_en)
        wr_ptr <= wr_ptr + 1;  // Increment write pointer
end

// Synchronizing the write pointer to the read clock domain
always @(posedge rd_clk or negedge rst) begin
    if (~rst) begin
        wr_ptr_sync1 <= 4'b0000;
```

```
wr_ptr_sync2 <= 4'b0000;
        end else begin
            wr_ptr_sync1 <= wr_ptr;
            wr_ptr_sync2 <= wr_ptr_sync1;
        end
    end

    // Read operation
    always @(posedge rd_clk or negedge rst) begin
        if (~rst)
            rd_ptr <= 4'b0000;   // Reset read pointer
        else if (rd_en)
            rd_ptr <= rd_ptr + 1;  // Increment read pointer
    end

    // Read data from memory
    always @(posedge rd_clk or negedge rst) begin
        if (~rst)
            rd_data <= 8'b0;
        else if (rd_en)
            rd_data <= mem[rd_ptr];
    end

    // Memory write operation
    always @(posedge wr_clk) begin
        if (wr_en)
            mem[wr_ptr] <= wr_data;  // Write data to memory
        end

endmodule
```

CASE-BASED INTERVIEW QUESTION 12

You are designing a priority encoder in Verilog. The priority encoder has 8 input signals and provides a 3-bit binary code as output corresponding to the highest-priority active input. If no inputs are active, the output should be invalid (3'bxxx). How would you implement this?

Asked in: Qualcomm, Apple, AMD, NVIDIA

SOLUTION STRATEGY:

1. **Use a priority checking mechanism to determine the highest-priority active input.**
2. **Implement a binary encoding for the highest-priority input.**
3. **Ensure that if no inputs are active, the output remains invalid.**

```
module priority_encoder (
    input wire [7:0] in,     // 8 input signals
    output reg [2:0] out,    // 3-bit binary code output
    output reg valid         // Valid signal to indicate if any input is active
);

    always @(*) begin
        case (in)
            8'b10000000: out = 3'b111;
            8'b01000000: out = 3'b110;
            8'b00100000: out = 3'b101;
            8'b00010000: out = 3'b100;
            8'b00001000: out = 3'b011;
            8'b00000100: out = 3'b010;
            8'b00000010: out = 3'b001;
            8'b00000001: out = 3'b000;
            default: out = 3'bxxx;   // No active input
        endcase
        valid = (in != 8'b00000000); // If any input is active, valid = 1
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 13

Design a frequency divider in Verilog that divides the input clock by a configurable integer value. The design should take an integer parameter for the division factor and output the divided clock signal.

Asked in: Broadcom, Intel, Qualcomm, Apple

SOLUTION STRATEGY:

1. **Use a counter to divide the input clock by the desired factor.**
2. **The division factor should be configurable using a parameter.**
3. **Ensure that the counter wraps around when the specified division value is reached.**

```
module frequency_divider #(
    parameter DIV_FACTOR = 4  // Division factor, default value 4
)(
    input wire clk,           // Input clock
    input wire rst,           // Active-low reset
    output reg divided_clk    // Divided clock output
);

    reg [31:0] counter;       // 32-bit counter to hold division count

    always @(posedge clk or negedge rst) begin
        if (~rst)
            counter <= 32'b0;
        else if (counter == (DIV_FACTOR - 1))
                counter <= 32'b0;             // Reset counter when division is
complete
        else
            counter <= counter + 1;    // Increment counter
    end
```

```
always @(posedge clk or negedge rst) begin
    if (~rst)
        divided_clk <= 1'b0;      // Reset divided clock
    else if (counter == (DIV_FACTOR - 1))
        divided_clk <= ~divided_clk;  // Toggle divided clock
end
endmodule
```

CASE-BASED INTERVIEW QUESTION 14

Design a pulse generator in Verilog that generates a 1-clock cycle wide pulse every time an input signal transitions from low to high (positive edge). This pulse should be synchronous to the input clock.

Asked in: AMD, NVIDIA, Qualcomm

SOLUTION STRATEGY:

1. **Detect the positive edge of the input signal.**
2. **Generate a 1-cycle pulse synchronized with the clock when the edge is detected.**
3. **Ensure proper synchronization and reset behavior.**

```
module pulse_generator (
    input wire clk,     // Input clock
    input wire rst,     // Active-low reset
    input wire signal,  // Input signal
    output reg pulse    // 1-clock cycle wide pulse
);
    reg signal_d; // Delayed signal

    always @(posedge clk or negedge rst) begin
        if (~rst)
            signal_d <= 1'b0;   // Reset delayed signal
        else
```

```
    else
        signal_d <= signal; // Capture the current signal value
end

always @(posedge clk or negedge rst) begin
    if (~rst)
        pulse <= 1'b0; // Reset pulse
    else
        pulse <= signal & ~signal_d; // Generate pulse on positive edge
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 15

You need to implement a deadlock detector for a system with two resources and two processes in a Verilog design. The system uses two process requests for two resources. You need to design a circuit that detects if the system has entered a deadlock state.

Asked in: Intel, AMD, Broadcom, Apple

SOLUTION STRATEGY:

1. **Create two sets of request signals, one for each process (Process 1 and Process 2).**
2. **Create allocation signals to indicate resource ownership.**
3. **Detect deadlock by checking if both processes are waiting for the resources held by each other.**

```
module deadlock_detector (
    input wire p1_req,  // Process 1 request for resource
    input wire p2_req,  // Process 2 request for resource
    input wire r1_alloc, // Resource 1 allocation signal
    input wire r2_alloc, // Resource 2 allocation signal
    output reg deadlock_detected  // Deadlock detected signal
);
```

```
    always @(*) begin
        // Check for deadlock condition: both processes waiting on each
other
        if (p1_req && p2_req && r1_alloc && r2_alloc)
            deadlock_detected = 1'b1;  // Deadlock detected
        else
            deadlock_detected = 1'b0;  // No deadlock
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 16

Design a state machine in Verilog for a simple elevator system. The elevator has three states: Idle, Moving Up, and Moving Down. The system should transition between these states based on button presses and floor requests.

Asked in: Qualcomm, AMD, NVIDIA, Apple

SOLUTION STRATEGY:

1. **Implement a state machine with three states: Idle, Moving Up, and Moving Down.**
2. **Use button presses and floor requests to trigger state transitions.**
3. **Ensure proper reset behavior.**

```
module elevator_fsm (
    input wire clk,         // Clock input
    input wire rst,         // Active-low reset
    input wire up_button,   // Up button press
    input wire down_button, // Down button press
    output reg [1:0] state  // Elevator state output
);
```

```
// State encoding
 parameter IDLE = 2'b00;
 parameter UP = 2'b01;
 parameter DOWN = 2'b10;

 // State transition logic
 always @(posedge clk or negedge rst) begin
     if (~rst)
         state <= IDLE;  // Reset to Idle state
     else begin
         case (state)
             IDLE: begin
                 if (up_button)
                     state <= UP;  // Move up if the up button is pressed
                 else if (down_button)
                     state <= DOWN;  // Move down if the down button is
pressed
             end
             UP: begin
                 if (~up_button)
                     state <= IDLE;  // Return to Idle if the up button is
released
             end
             DOWN: begin
                 if (~down_button)
                     state <= IDLE;  // Return to Idle if the down button is
released
             end
         endcase
     end
 end

endmodule
```

CASE-BASED INTERVIEW QUESTION 17

Design a timing constraint checker in Verilog to verify that the setup and hold time requirements for flip-flops are met. The checker should provide an output signal that indicates if there is a setup violation or a hold violation.

Asked in: Intel, NVIDIA, Qualcomm, AMD

SOLUTION STRATEGY:

1. **Use delay elements to simulate the propagation delay of signals.**
2. **Check the setup and hold times against the flip-flop's operating conditions.**
3. **Generate violation flags based on the timing analysis.**

```
module timing_checker (
    input wire clk,          // Clock input
    input wire rst,          // Active-low reset
    input wire d,            // Data input
    input wire q,            // Flip-flop output
    input wire clk_edge,     // Clock edge
    output reg setup_violation, // Setup violation output
    output reg hold_violation  // Hold violation output
);

    reg [1:0] d_delayed;

    // Check setup and hold violations
    always @(posedge clk or negedge rst) begin
        if (~rst) begin
            d_delayed <= 2'b00;
            setup_violation <= 1'b0;
            hold_violation <= 1'b0;
        end else begin
            // Simulate setup and hold violation detection logic
            d_delayed <= {d_delayed[0], d};  // Delay the data signal
```

```
// Check for setup violation (if data is stable before clock edge)
if (clk_edge && (d != d_delayed[1]))
    setup_violation <= 1'b1;
else
    setup_violation <= 1'b0;

// Check for hold violation (if data is not stable after clock
edge)
        if (~clk_edge && (d != d_delayed[0]))
            hold_violation <= 1'b1;
        else
            hold_violation <= 1'b0;
      end
   end

endmodule
```

CASE-BASED INTERVIEW QUESTION 18

You need to design a digital clock that displays the time in hours (00-23), minutes (00-59), and seconds (00-59). The clock is driven by a 1 Hz clock signal. The design should handle overflow correctly (i.e., 59 seconds should increment the minute counter, and 59 minutes should increment the hour counter). The clock should also handle reset functionality.

Asked in: Intel, NVIDIA, Apple, Broadcom

SOLUTION STRATEGY:

1. **Use a counter-based approach to count seconds, minutes, and hours.**
2. **Implement overflow logic for seconds and minutes.**
3. **Use reset to initialize the clock to 00:00:00.**

```
module digital_clock (
    input wire clk,          // 1 Hz clock input
    input wire rst,          // Active-low reset
    output reg [5:0] hours,  // Hours (00-23)
    output reg [5:0] minutes,// Minutes (00-59)
    output reg [5:0] seconds // Seconds (00-59)
);

// State registers
reg [5:0] sec_cnt;    // Seconds counter (0-59)
reg [5:0] min_cnt;    // Minutes counter (0-59)
reg [5:0] hour_cnt;   // Hours counter (0-23)

// Reset logic and counter update
always @(posedge clk or negedge rst) begin
    if (~rst) begin
        sec_cnt <= 6'b000000;    // Reset seconds to 0
        min_cnt <= 6'b000000;    // Reset minutes to 0
        hour_cnt <= 6'b000000;   // Reset hours to 0
    end else begin
        // Increment seconds
        if (sec_cnt == 6'b111011) begin // 59 seconds
            sec_cnt <= 6'b000000;    // Reset seconds to 0
            // Increment minutes if seconds overflow
            if (min_cnt == 6'b111011) begin // 59 minutes
                min_cnt <= 6'b000000;    // Reset minutes to 0
                // Increment hours if minutes overflow
                if (hour_cnt == 6'b010111) begin // 23 hours
                    hour_cnt <= 6'b000000;    // Reset hours to 0 (overflow)
                end else begin
                    hour_cnt <= hour_cnt + 1; // Increment hours
                end
            end else begin
                min_cnt <= min_cnt + 1;    // Increment minutes
```

```
            end
        end else begin
            sec_cnt <= sec_cnt + 1;    // Increment seconds
        end
    end
end

    // Assign outputs
    always @(*) begin
        hours = hour_cnt;
        minutes = min_cnt;
        seconds = sec_cnt;
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 19

Design a reliable communication protocol using handshaking between two modules in Verilog. The sender module sends data in a FIFO buffer to the receiver module, which reads it at its own pace. The design should include acknowledgment signals, overflow handling, and underflow detection.

Asked in: Qualcomm, AMD, Nvidia, Intel

SOLUTION STRATEGY:

1. **Implement handshaking using the valid and ack signals to synchronize the sender and receiver.**
2. **Use a FIFO buffer for storing data.**
3. **Handle overflow when the sender's data is not consumed quickly enough by the receiver and underflow when the receiver reads from an empty FIFO.**

```
module fifo_handshaking (
    input wire clk,              // Clock input
    input wire rst,              // Active-low reset
    input wire [7:0] data_in,    // Data input to FIFO
    input wire valid_in,         // Valid signal from sender
    output reg ack_in,           // Acknowledge signal from receiver
    output reg [7:0] data_out,   // Data output from FIFO
    output reg valid_out,        // Valid signal for receiver
    input wire ack_out           // Acknowledge signal from receiver
);

    // FIFO buffer
    reg [7:0] fifo [15:0];   // 16-deep FIFO for 8-bit data
    reg [3:0] write_ptr, read_ptr;
    reg full, empty;

    always @(posedge clk or negedge rst) begin
    if (~rst) begin
        write_ptr <= 4'b0000; // Reset write pointer
        read_ptr <= 4'b0000;  // Reset read pointer
        full <= 1'b0;
        empty <= 1'b1;
        ack_in <= 1'b0;
        valid_out <= 1'b0;
    end else begin
        // FIFO write operation with handshaking
        if (valid_in && !full) begin
        fifo[write_ptr] <= data_in;  // Store data in FIFO
        write_ptr <= write_ptr + 1;  // Increment write pointer
        full <= (write_ptr == 4'b1111);  // Check for overflow
        empty <= 1'b0;
        ack_in <= 1'b1;  // Acknowledge that data is written
    end else begin
        ack_in <= 1'b0;
    end
```

```
// FIFO read operation with handshaking
if (ack_out && !empty) begin
    data_out <= fifo[read_ptr];  // Output data from FIFO
    read_ptr <= read_ptr + 1;    // Increment read pointer
    empty <= (read_ptr == 4'b1111);  // Check for underflow
    valid_out <= 1'b1;  // Indicate data is valid for receiver
end else begin
            valid_out <= 1'b0;
        end
    end
  end

endmodule
```

CASE-BASED INTERVIEW QUESTION 20

Design a circular shift register in Verilog that performs left and right shifts. The shift direction should be controlled by an input signal, and the register should handle reset correctly.

Asked in: Broadcom, AMD, Nvidia, Intel

SOLUTION STRATEGY:

1. **Use a shift register array to store data.**
2. **Implement left and right shift functionality based on the control signal.**
3. **Handle reset condition for initializing the shift register.**

```
module circular_shift_register (
    input wire clk,          // Clock input
    input wire rst,          // Active-low reset
    input wire [7:0] data_in, // Data input to the shift register
    output reg [7:0] data_out,// Data output from the shift register
    input wire shift_left,    // Shift left control
    input wire shift_right    // Shift right control
);
```

```
reg [7:0] shift_reg;  // 8-bit shift register

always @(posedge clk or negedge rst) begin
    if (~rst) begin
        shift_reg <= 8'b00000000;  // Reset shift register to 0
    end else if (shift_left) begin
        shift_reg <= {shift_reg[6:0], shift_reg[7]};  // Left shift
    end else if (shift_right) begin
        shift_reg <= {shift_reg[0], shift_reg[7:1]};  // Right shift
    end
end

// Output the current state of the shift register
always @(*) begin
    data_out = shift_reg;
end

endmodule
```

CASE-BASED INTERVIEW QUESTION 21

Design a parametrizable N-bit pipelined multiplier in Verilog that multiplies two N-bit unsigned numbers. The pipeline stages must register partial results to increase throughput. Provide support for reset and valid signals.

Asked in: Intel, NVIDIA, AMD, Synopsys

SOLUTION STRATEGY:

1. **Create a pipelined architecture with N stages.**
2. **Use registers to store partial products at each stage.**
3. **Support valid/ready handshaking and reset.**

```
module pipelined_multiplier #(
    parameter N = 8
)(
    input wire clk,
    input wire rst,
    input wire valid_in,
    input wire [N-1:0] a,
    input wire [N-1:0] b,
    output reg valid_out,
    output reg [2*N-1:0] product
);

    // Pipeline registers
    reg [2*N-1:0] pipe_reg [0:N-1];
    reg [N-1:0] a_reg [0:N-1];
    reg [N-1:0] b_reg;
    reg valid_pipe [0:N-1];

integer i;
always @(posedge clk or negedge rst) begin
    if (!rst) begin
        for (i = 0; i < N; i = i + 1) begin
            pipe_reg[i] <= 0;
            a_reg[i] <= 0;
            valid_pipe[i] <= 0;
        end
        b_reg <= 0;
        product <= 0;
        valid_out <= 0;
    end else begin
        if (valid_in) begin
            a_reg[0] <= a;
            b_reg <= b;
            pipe_reg[0] <= a[0] ? b : 0;
            valid_pipe[0] <= 1;
```

```
    end

    for (i = 1; i < N; i = i + 1) begin
        pipe_reg[i] <= pipe_reg[i-1] + (a_reg[i-1][i] ? (b_reg << i) : 0);
        a_reg[i] <= a_reg[i-1];
        valid_pipe[i] <= valid_pipe[i-1];
    end

            product <= pipe_reg[N-1];
            valid_out <= valid_pipe[N-1];
        end
    end

endmodule
```

CASE-BASED INTERVIEW QUESTION 22

Design a parameterizable UART transmitter and receiver module in Verilog. The transmitter must send data serially with start and stop bits. The receiver should detect framing errors and support configurable baud rates.

Asked in: Intel, NVIDIA, AMD, Synopsys

SOLUTION STRATEGY:

1. **Use FSM for transmitter and receiver.**
2. **Support configurable BAUD_TICK count for baud rate.**
3. **Add error detection for frame issues (missing stop/start bits).**

```
module uart_tx (
    input wire clk,
    input wire rst,
    input wire [7:0] tx_data,
    input wire tx_start,
    output reg tx_line,
    output reg tx_busy
);
```

```
reg [3:0] state;
reg [3:0] bit_cnt;
reg [9:0] tx_shift;

always @(posedge clk or negedge rst) begin
    if (!rst) begin
        tx_line <= 1'b1;
        tx_busy <= 1'b0;
        state <= 0;
    end else begin
        case (state)
            0: if (tx_start) begin
                tx_shift <= {1'b1, tx_data, 1'b0};  // stop + data + start
                bit_cnt <= 0;
                tx_busy <= 1;
                state <= 1;
            end
            1: begin
                tx_line <= tx_shift[bit_cnt];
                bit_cnt <= bit_cnt + 1;
                if (bit_cnt == 9) begin
                    state <= 2;
                end
            end
            2: begin
                tx_busy <= 0;
                state <= 0;
            end
        endcase
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 23

Design a 4-way Set-Associative Cache in Verilog with LRU replacement policy. Each cache line holds a 32-bit word. Implement:

- **Cache hit/miss detection**
- **Tag comparison**
- **LRU tracking**

Asked in: Intel, NVIDIA, Apple, AMD

SOLUTION STRATEGY:

1. **Use associative tag arrays with 4 blocks per set**
2. **Track usage for LRU with shift register or counter**
3. **Output hit/miss and read data**

```
module set_assoc_cache #(
    parameter INDEX_BITS = 4,
    parameter TAG_BITS = 8
)(
    input clk,
    input rst,
    input [TAG_BITS + INDEX_BITS - 1:0] addr,
    input [31:0] write_data,
    input wr_en,
    output reg [31:0] read_data,
    output reg hit
);

    localparam SETS = 1 << INDEX_BITS;

    reg [TAG_BITS-1:0] tag_array [0:3][0:SETS-1];
    reg valid_array [0:3][0:SETS-1];
    reg [31:0] data_array [0:3][0:SETS-1];
    reg [1:0] lru [0:SETS-1];  // 2-bit LRU indicator
```

```
    wire [INDEX_BITS-1:0] index = addr[INDEX_BITS-1:0];
    wire [TAG_BITS-1:0] tag = addr[TAG_BITS + INDEX_BITS -
1:INDEX_BITS];

    integer i;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
        for (i = 0; i < SETS; i = i + 1) begin
            valid_array[0][i] <= 0;
            valid_array[1][i] <= 0;
            valid_array[2][i] <= 0;
            valid_array[3][i] <= 0;
            lru[i] <= 0;
        end
        hit <= 0;
    end else begin
        hit <= 0;
        for (i = 0; i < 4; i = i + 1) begin
            if (valid_array[i][index] && tag_array[i][index] == tag) begin
                read_data <= data_array[i][index];
                hit <= 1;
                lru[index] <= i;  // update LRU
            end
        end

        if (wr_en && !hit) begin
            // Replace LRU
                tag_array[lru[index]][index] <= tag;
                data_array[lru[index]][index] <= write_data;
                valid_array[lru[index]][index] <= 1;
                lru[index] <= (lru[index] + 1) % 4;  // round robin LRU
            end
        end
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 24

Design a DDR Memory Interface Controller (read-only) in Verilog. Your controller should:

- **Read burst of 4 data words**
- **Handle clock latency (CAS latency simulation)**
- **Align data properly with command latency**

Asked in: Micron, Samsung, Intel, Marvell

SOLUTION STRATEGY:

1. **FSM with states: IDLE, ACTIVATE, READ_CMD, WAIT, READ_BURST**
2. **Simulate burst-read alignment and latency**
3. **Use shift register to simulate CAS latency**

```
module ddr_read_ctrl (
    input clk,
    input rst,
    input read_req,
    input [15:0] addr,
    output reg [31:0] data_out,
    output reg valid
);

    parameter LATENCY = 3;
    reg [2:0] state;
    reg [2:0] latency_cnt;
    reg [31:0] mem [0:65535];
    reg [1:0] burst_cnt;
    reg [15:0] read_addr;

    localparam IDLE = 0,
               ACTIVATE = 1,
               WAIT_LAT = 2,
               BURST_READ = 3;
```

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
        valid <= 0;
    end else begin
        case (state)
            IDLE: begin
                valid <= 0;
                if (read_req) begin
                    read_addr <= addr;
                    state <= ACTIVATE;
                end
            end
            ACTIVATE: begin
                latency_cnt <= LATENCY;
                state <= WAIT_LAT;
            end
            WAIT_LAT: begin
                if (latency_cnt == 0) begin
                    burst_cnt <= 0;
                    state <= BURST_READ;
                end else begin
                    latency_cnt <= latency_cnt - 1;
                end
            end
            BURST_READ: begin
                data_out <= mem[read_addr + burst_cnt];
                valid <= 1;
                burst_cnt <= burst_cnt + 1;
                if (burst_cnt == 3)
                    state <= IDLE;
            end
        endcase
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 25

Design an Out-of-Order Instruction Reorder Buffer (ROB) that supports:

- **Tagging issued instructions**
- **Committing them in order**
- **Handling exceptions and flushes**

Asked in: Apple, AMD, Arm, Intell

SOLUTION STRATEGY:

1. **Circular buffer with head/tail pointers**
2. **Maintain validity and tags for each entry**
3. **Commit in order from head**
4. **Support flush on exception**

```
module reorder_buffer #(
    parameter WIDTH = 8,
    parameter TAG_WIDTH = 4
)(
    input clk,
    input rst,
    input issue,
    input commit,
    input [TAG_WIDTH-1:0] tag_in,
    input exception,
    output reg [TAG_WIDTH-1:0] tag_out,
    output reg valid_out
);

reg [TAG_WIDTH-1:0] rob_tags [0:WIDTH-1];
reg valid [0:WIDTH-1];

reg [2:0] head, tail;
```

```
integer i;
always @(posedge clk or posedge rst) begin
    if (rst) begin
        head <= 0;
        tail <= 0;
        valid_out <= 0;
        for (i = 0; i < WIDTH; i = i + 1)
            valid[i] <= 0;
    end else if (exception) begin
        head <= 0;
        tail <= 0;
        for (i = 0; i < WIDTH; i = i + 1)
            valid[i] <= 0;
    end else begin
        if (issue) begin
            rob_tags[tail] <= tag_in;
            valid[tail] <= 1;
            tail <= (tail + 1) % WIDTH;
        end
        if (commit && valid[head]) begin
            tag_out <= rob_tags[head];
            valid_out <= 1;
            valid[head] <= 0;
            head <= (head + 1) % WIDTH;
        end else begin
            valid_out <= 0;
        end
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 26

Design a Multi-Channel DMA Controller supporting:

- **4 independent channels**
- **Memory-to-memory transfers**
- **Priority-based arbitration**
- **Interrupt generation on completion**

Asked in: Qualcomm, Broadcom, Marvell, Intel

SOLUTION STRATEGY:

1. **Use FSMs for transfer logic per channel**
2. **Priority encoder to choose which channel runs**
3. **Use AXI-lite or simple interface for memory**

```
module dma_controller (
    input clk,
    input rst,
    input [3:0] start,          // Start signal per channel
    input [31:0] src_addr[3:0],
    input [31:0] dest_addr[3:0],
    input [15:0] length[3:0],
    output reg [3:0] done       // Done signal per channel
);

    reg [1:0] current_channel;
    reg [15:0] count;
    reg busy;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        busy <= 0;
        done <= 0;
    end else begin
        if (!busy) begin
```

```
// Priority-based arbitration
if (start[0]) current_channel <= 0;
else if (start[1]) current_channel <= 1;
else if (start[2]) current_channel <= 2;
else if (start[3]) current_channel <= 3;

if (|start) begin
    busy <= 1;
    count <= length[current_channel];
  end
end else begin
  // Simulate transfer
  if (count > 0) begin
    count <= count - 1;
  end else begin
    done[current_channel] <= 1;
    busy <= 0;
      end
    end
  end
end
endmodule
```

CASE-BASED INTERVIEW QUESTION 27

Design a Systolic Array for Matrix Multiplication (4x4) using registers and multiply-accumulate units. Focus on:

- **Data flow pipelining**
- **Local communication**
- **Parallelism**

Asked in: Google, Apple, Tesla, Tenstorrent

SOLUTION STRATEGY:

1. **Build processing elements (PEs) in grid**
2. **Each PE computes: C(i,j) += A(i,k) * B(k,j)**
3. **Use shift-register style data movement**

```
module pe (
    input clk,
    input rst,
    input [7:0] a_in,
    input [7:0] b_in,
    input [15:0] c_in,
    output reg [7:0] a_out,
    output reg [7:0] b_out,
    output reg [15:0] c_out
);
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            a_out <= 0;
            b_out <= 0;
            c_out <= 0;
        end else begin
            c_out <= c_in + a_in * b_in;
            a_out <= a_in;
            b_out <= b_in;
        end
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 28

Design an AXI4-Lite Slave Interface to read/write internal registers.
Include:

- **Read/write transactions**
- **Ready/valid handshaking**
- **Address decode for 4 registers**

Asked in: Xilinx, Intel, AMD, MediaTek

SOLUTION STRATEGY:

1. **Implement AXI handshaking rules**
2. **Use awvalid/awready, wvalid/wready, bvalid/bready, etc.**
3. **Decode address into internal registers**

```
module axi_lite_slave (
    input clk,
    input rst,
    input [3:0] awaddr,
    input awvalid,
    output reg awready,

    input [31:0] wdata,
    input wvalid,
    output reg wready,

    output reg bvalid,
    input bready,

    input [3:0] araddr,
    input arvalid,
    output reg arready,

    output reg [31:0] rdata,
    output reg rvalid,
    input rready
);
```

```
reg [31:0] regs [0:3];

always @(posedge clk or posedge rst) begin
    if (rst) begin
        awready <= 0; wready <= 0; bvalid <= 0;
        aready <= 0; rvalid <= 0;
    end else begin
    // Write Address
    if (awvalid && !awready) awready <= 1;
    else awready <= 0;

    // Write Data
    if (wvalid && !wready) begin
        regs[awaddr[3:2]] <= wdata;
        wready <= 1;
        bvalid <= 1;
    end else wready <= 0;

    if (bvalid && bready) bvalid <= 0;

    // Read
    if (arvalid && !aready) begin
        rdata <= regs[araddr[3:2]];
        aready <= 1;
        rvalid <= 1;
    end else begin
        aready <= 0;
            if (rvalid && rready) rvalid <= 0;
        end
      end
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 29

Design a Pipelined CPU Datapath supporting:

- **5-stage pipeline: IF, ID, EX, MEM, WB**
- **Data forwarding to handle RAW hazards**
- **Stall mechanism for load-use hazards**

Asked in: Apple, Tesla, NVIDIA, AMD

SOLUTION STRATEGY:

1. **Separate logic for each pipeline stage**
2. **Use forwarding muxes to bypass data**
3. **Detect hazards and insert NOPs/stalls**

```
module ex_stage (
    input [31:0] reg_rs1,
    input [31:0] reg_rs2,
    input [31:0] ex_mem_result,
    input [31:0] mem_wb_result,
    input [1:0] forward_a,
    input [1:0] forward_b,
    input [3:0] alu_op,
    output reg [31:0] alu_result
);

    reg [31:0] operand_a, operand_b;

    always @(*) begin
    case (forward_a)
        2'b00: operand_a = reg_rs1;
        2'b01: operand_a = mem_wb_result;
        2'b10: operand_a = ex_mem_result;
        default: operand_a = 32'b0;
    endcase
```

```
case (forward_b)
    2'b00: operand_b = reg_rs2;
    2'b01: operand_b = mem_wb_result;
    2'b10: operand_b = ex_mem_result;
    default: operand_b = 32'b0;
endcase
```

```
        case (alu_op)
            4'b0000: alu_result = operand_a + operand_b;
            4'b0001: alu_result = operand_a - operand_b;
            4'b0010: alu_result = operand_a & operand_b;
            4'b0011: alu_result = operand_a | operand_b;
            default: alu_result = 32'b0;
        endcase
    end
endmodule
```

CASE-BASED INTERVIEW QUESTION 30

Design a Triple Modular Redundancy (TMR) Logic Block to improve fault tolerance for critical applications.

The circuit should:

- **Replicate logic 3 times**
- **Use majority voting for output**
- **Detect faulty units**

Asked in: SpaceX, Intel, Boeing, Honeywell Aerospace

SOLUTION STRATEGY:

1. **Use 3 identical logic modules**
2. **Feed outputs to a majority voter**
3. **Optionally log faults if any module differs**

```
module tmr_logic (
    input [3:0] in,
    output reg out,
    output reg fault_detected
);

    wire out1, out2, out3;

logic_block u1 (.in(in), .out(out1));
logic_block u2 (.in(in), .out(out2));
logic_block u3 (.in(in), .out(out3));

always @(*) begin
    // Majority voter
    out = (out1 & out2) | (out2 & out3) | (out1 & out3);
    // Fault detection if outputs are not all equal
    fault_detected = (out1 != out2) || (out2 != out3) || (out1 != out3);
    end
endmodule

module logic_block (
    input [3:0] in,
    output out
);
    assign out = (in[3] & ~in[2]) | (in[1] ^ in[0]);
endmodule
```

PROV LOGIC

Excellence in World class VLSI Training & Placements

Do follow for updates & enquires

+91- 9182280927